



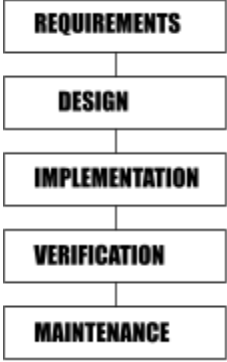
Republic of the Philippines
ONE CAINTA COLLEGE
Bachelor of Science in Information Systems

SDLC ACTIVITY

A Project/Assignment/Activity in Database Administration
Presented to the faculty of One Cainta College
Cainta, Rizal

In partial fulfillment of the requirements for
Bachelor of Science in Information Systems

Submitted by:
Emerson T. Jacinto

NAME	DESCRIPTION	ILLUSTRATION	ADVANTAGES	DISADVANTAGES	SYSTEM APPLICATION
WATERFALL MODEL	-The oldest and most straightforward of the structured SDLC methodologies. -Each phase must be completed before moving on to the next, with no returning to previous stages. -This model is not ideal when flexibility is required or for long-term, evolving projects.	 <pre> graph TD A[REQUIREMENTS] --> B[DESIGN] B --> C[IMPLEMENTATION] C --> D[VERIFICATION] D --> E[MAINTENANCE] </pre>	-Simple and easy to understand and implement. -Phases are completed sequentially, one at a time. -Works best for smaller projects with clearly defined and well-understood requirements.	-No working software is produced until the later stages of the life cycle. -Not well suited for complex or object-oriented projects. -Progress can be difficult to measure within each phase.	-Best suited for small or simple projects where requirements are stable, clearly defined, and unlikely to change.

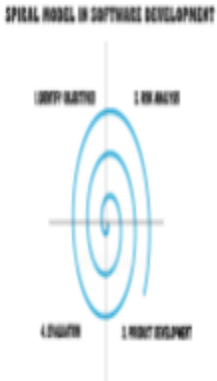
WATERFALL MODEL

NAME	DESCRIPTION	ILLUSTRATION	ADVANTAGES	DISADVANTAGES	SYSTEM APPLICATION
AGILE MODEL	-Emphasizes that each project should be managed using an approach tailored to its specific needs. -Tasks are divided into time boxes (short, fixed periods) to deliver specific features for each release.	<pre>graph TD; PLAN[1. PLAN] --> LAUNCH[1. LAUNCH]; PLAN --> DESIGN[2. DESIGN]; LAUNCH --> REVIEW[6. REVIEW]; REVIEW --> DEPLOY[5. DEPLOY]; DESIGN --> DEVELOP[3. DEVELOP]; DEVELOP --> TEST[4. TEST]; DEPLOY <--> TEST;</pre>	-Easily adapts to changing requirements, even in later stages of development. -Delivers small increments of working software quickly. -Continuous testing in each sprint helps reduce defects.	-It is more difficult to accurately predict time, cost, and scope at the beginning of the project. -Emphasis on working software may result in less detailed documentation. -Continuous changes can lead to scope creep or uncontrolled feature expansion.	-Ideal for web applications and e-commerce platforms. -Suitable for mobile applications that require frequent feature releases.

AGILE MODEL

NAME	DESCRIPTION	ILLUSTRATION	ADVANTAGES	DISADVANTAGES	SYSTEM APPLICATION
ITERATIVE MODEL	-This model provides a working version early in the process, making changes easier and less costly to implement.	<pre>graph TD; INIT[INITIALIZATION] --> P1[PLANNING]; P1 --> RG[REQUIREMENTS GATHERING]; RG --> D[DESIGN]; D --> C[CODING]; C --> DE[DEPLOYMENT]; P1 --> E[EVALUATION]; E --> P2[PLANNING]; P2 --> C;</pre>	-Software is developed quickly throughout the life cycle. -The model is flexible, making it easier and less expensive to change requirements and scope. -It is generally more cost-effective compared to other models.	-Each phase of an iteration is rigid, with no overlap between phases. -Issues with system architecture may occur because not all requirements are gathered upfront for the entire software life cycle.	-Suitable for web applications where user requirements may evolve during development. -Ideal for mobile apps, particularly startups testing MVPs. -Enterprise software modules can be delivered incrementally to gather user feedback.

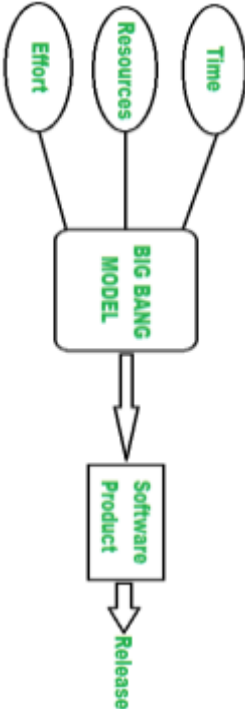
ITERATIVE MODEL

NAME	DESCRIPTION	ILLUSTRATION	ADVANTAGES	DISADVANTAGES	SYSTEM APPLICATION
SPIRAL MODEL	-One of the most flexible SDLC methodologies. -Allows building a highly customized product, incorporating user feedback from the early stages of the project.		-Identifies and mitigates risks early through continuous assessment. -Allows changes to be incorporated at any phase due to its iterative nature. -Detects risks and design flaws before major costs are incurred.	-Risk analysis and iterative cycles make it more expensive than simpler models. -Requires careful planning and experienced project management. -Multiple iterations can extend the overall development timeline.	-Suitable for large-scale software projects with high complexity and critical risks. -Ideal for banking and finance systems, where reliability and risk mitigation are crucial. -Appropriate for projects with unclear or evolving requirements that benefit from iterative refinement.

SPIRAL MODEL

NAME	DESCRIPTION	ILLUSTRATION	ADVANTAGES	DISADVANTAGES	SYSTEM APPLICATION
V MODEL	-Also known as the Verification and Validation . -Evolved from the Waterfall model and features a corresponding testing phase for each development stage. -Each stage begins only after the previous stage has been completed.	<pre> graph TD RA[Requirements Analysis] --> SD[System Design] SD --> AD[Architecture Design] AD --> MD[Module Design] MD --> C[Coding] C --> UT[Unit Testing] UT --> IT[Integration Testing] IT --> ST[System Testing] ST --> AT[Acceptance Testing] </pre>	-A highly disciplined model where phases are completed sequentially, one at a time. -Works well for smaller projects with clearly defined and well-understood requirements. -Simple and easy to understand and implement.	-Involves high risk and uncertainty. -Not suitable for complex or object-oriented projects. -Unsuitable for long-term or ongoing projects.	-Suitable for medical software systems, such as patient monitoring, where safety and accuracy are critical. -Ideal for banking and financial applications that require strict verification and validation of transactions.

V MODEL

NAME	DESCRIPTON	ILLUSTRATION	ADVANTAGES	DISADVANTAGES	SYSTEM APPLICATION
BIGBANG MODEL	-Focuses primarily on software development and coding, with minimal or no formal planning. -Ideal for small projects with one or two developers and also suitable for academic or practice projects.	 <pre>graph TD; E([Effort]) --> B[\"BIG BANG MODEL\"]; R([Resources]) --> B; T([Time]) --> B; B --> P[\"Software Product\"]; P --> R2[Release];</pre>	-A very simple model with minimal or no planning required. -Easy to manage and implement. -Requires very few resources.	-Involves very high risk and uncertainty. -Unsuitable for complex or object-oriented projects. -Not recommended for long-term or ongoing projects.	-Suitable for prototypes or quick proof-of-concept applications. -Ideal for small-scale or personal projects, such as simple apps or utilities. -Works well for short-term programs or small tools with few modules and low complexity.

BIGBANG MODEL

NAME	DESCRIPTION	ILLUSTRATION	ADVANTAGES	DISADVANTAGES	SYSTEM APPLICATION
RAD MODEL	-Emphasizes gathering customer requirements through workshops or focus groups. -Relies on prototyping and iterative development, with minimal formal planning. -Prioritizes rapid prototyping over extensive upfront planning.	<pre> graph TD BM[Business Modeling] --> DM[Data Modeling] DM --> PM[Process Modeling] PM --> AG[Application Generation] AG --> TT[Testing & Turnover] TT --> BM </pre>	-Systems can be developed and delivered quickly. -Requirements can be easily modified during development. -Errors are detected early through prototyping and testing.	-Difficult to manage for very complex systems. -May require additional resources and specialized tools. -Emphasizes speed over detailed documentation.	-Suitable for small to medium-sized business systems. -Ideal for web-based applications. -Can be used for student information systems.

RAD MODEL